

Discovery Executive Summary

Project: discovery-27july2026 · Generated: 27/07/2026, 12:56:52

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Testing & Quality Assurance Analysis	HIGH RISK	—

1. Testing & Quality Assurance Analysis

OVERALL CODEBASE RATING — TESTING & QUALITY ASSURANCE

High Risk

Driven by zero-tested critical logic (H1), ~2% coverage (H2), and the total absence of integration, contract, frontend and E2E tests (H3, H4, H7).

EXECUTIVE SUMMARY

The repository ships with a test suite that exists in name only: **2 PHPUnit files** guard a codebase of **36 source files** across three layers (Laravel backend, Node/Express `dev-api`, React frontend). Both existing tests are vacuous — one asserts a hard-coded literal array equals itself, the other loops over `random_int()` and asserts a probabilistic threshold — so the **effective coverage of real business logic is ~2% (estimated)**; there is no coverage tooling (`coverage: none` in CI) to measure it. Every business-critical module is untested: the IVR-traversal / reachability engine (`RealTimeTestService`, `dev-api/src/realtime.js`), the Mongo persistence layer (`MongoService`), the unsafe-`extract()` `LegacyDataMapper`, and all six API controllers exposing ~18 endpoints. There are **no integration tests** exercising the DB/Mongo/HTTP/SSE boundaries, **no contract tests** for any public API, and **no frontend or end-to-end tests at all** despite a React app with routing, forms, and live SSE feeds. CI runs `phpunit` and a frontend `build` on every PR, but with the suite this thin the gate provides false confidence rather than protection. Overall testing health is **High Risk** and must be raised before any refactor or extraction work begins.

5.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Untested Critical Logic	Critical modules with zero tests	0	1–3	>3	6+ (RealTimeTestService, MongoService, LegacyDataMapper, ConnectController, DiscoveryController, dev-api/realtime.js)	HIGH RISK
H2	Low Test Coverage	Overall coverage %	>80%	50–80%	<50%	~2% (est., no coverage tool)	HIGH RISK
H3	Missing Integration Tests	Boundaries covered %	>70%	30–70%	<30%	0% (no Feature/HTTP/DB/SSE tests)	HIGH RISK
H4	Missing Contract Tests	APIs with contract tests %	>80%	40–80%	<40%	0% (~18 endpoints, 0 covered)	HIGH RISK
H5	Flaky / Skipped Tests	Skipped/flaky test count	0	1–5	>5	1 (non-deterministic <code>random_int</code> test)	MODERATE
H6	No CI Test Gate	Tests enforced in CI	Required gate	Runs, not required	No CI test run	Backend phpunit runs on PR (not proven required); frontend runs no tests	MODERATE
H7	No End-to-End Tests (additional)	Critical user flows with E2E coverage %	>60%	20–60%	<20%	0% (no Cypress/Playwright)	HIGH RISK
H8	Assertion-Free / Vacuous Tests (additional)	Tests with meaningful assertions %	>90%	50–90%	<50%	0% (0 of 2 assert real logic)	HIGH RISK

5.4 Actions Required

Hotspot	Action	Rating	Priority
H1 Untested Critical Logic	Add PHPUnit unit tests for <code>RealTimeTestService</code> , the reachability calc, <code>LegacyDataMapper</code> , <code>MongoService</code> ; mirror in <code>dev-api</code>	HIGH RISK	CRITICAL
H2 Low Test Coverage	Enable coverage tooling per layer; drive critical modules to 75–80% before refactors	HIGH RISK	CRITICAL

H3 Missing Integration Tests	Add Laravel feature tests (<code>RefreshDatabase</code>) + Mongo round-trip (<code>mongodb-memory-server</code>) + SSE smoke tests	HIGH RISK	HIGH
H4 Missing Contract Tests	Add <code>assertJsonStructure</code> tests for all ~18 API routes; validate frontend client against <code>types/index.ts</code>	HIGH RISK	HIGH
H7 No End-to-End Tests	Add Playwright happy-path specs per module (discovery + connect flows), run against <code>dev-api</code> in CI	HIGH RISK	HIGH
H8 Vacuous Tests	Replace both tautological tests with real assertions; add mutation testing (Infection)	HIGH RISK	HIGH
H5 Flaky Test	Delete the <code>random_int</code> reachability test; ban randomness in <code>tests/</code> ; replace with deterministic case	MODERATE	MEDIUM
H6 No CI Test Gate	Add frontend test step, enable coverage, make test jobs required branch-protection checks	MODERATE	MEDIUM

5.5 Expected Outcomes

- **Critical telecom logic is protected before refactors** — IVR traversal, reachability scoring, and the unsafe `LegacyDataMapper` gain real unit tests, so extraction/refactor work has a behavioural safety net.
- **Measurable coverage replaces guesswork** — per-layer coverage instruments turn "~2% estimated" into an enforced, ratcheting number on the path to 75–80%.
- **Boundaries are verified, not assumed** — integration tests exercise the controller → MariaDB → Mongo → SSE wiring, catching failures that unit tests miss.
- **API contracts can't drift silently** — contract tests on all ~18 endpoints (and the frontend client) fail the build the moment a response shape changes, protecting the customer dashboard.
- **The frontend gains a safety net** — Vitest unit tests plus Playwright E2E cover the React flows and live SSE feed that today have zero automated verification.
- **CI becomes a real gate** — a required, coverage-aware check across backend, frontend, and E2E means regressions are blocked at merge instead of discovered in production.

The complete report (including §5.2 Hotspot-by-Hotspot Evidence with `affected-files` directives and §5.3 Mermaid diagrams) is saved at `docs/discovery/05-testing-and-quality-assurance.md`, ready for the UI to render to PDF.